

Vertex MCP Bible

*VIC: ingeniería de un núcleo de ciberseguridad autónoma
con MCP, LangGraph y modelos locales*

Denis Sanchez Leyva

Vertex Coders LLC

© 2026 Denis Sanchez Leyva / Vertex Coders LLC, Miami, FL.

Todos los derechos reservados. Ninguna parte de esta publicacion puede ser reproducida sin autorizacion previa del autor.

Vertex MCP Bible

Primera edicion, 2026.

Prefacio

En las trincheras de la ingeniería de élite, la diferencia entre la victoria y el desastre no radica en las herramientas disponibles, sino en cómo se orquestan y blindan. Vertex MCP Bible nace de una necesidad táctica ineludible: la urgencia de automatizar la ciberseguridad ofensiva y la auditoría de infraestructuras sin ceder la soberanía de los datos a las nubes centralizadas.

Durante el desarrollo del Vertex Intelligence Core (VIC), enfrentamos obstáculos que los manuales convencionales no contemplaban. Desde las restricciones de virtualización de Docker en Windows hasta los nuevos vectores de ataque introducidos por la Inyección de Prompts en LLMs. Este libro no es un compendio teórico; es el diario de batalla y la prueba viviente de que es posible construir un cerebro neuronal local, autónomo y air-gapped.

Lo que tienen en sus manos es el estándar operativo oficial de Vertex Coders LLC, diseñado para empoderar al ingeniero moderno en la construcción del futuro de la inteligencia artificial desde sus propias trincheras.

¿Para quién es este libro?

Este manual está dirigido a perfiles técnicos avanzados que buscan cruzar la frontera entre la ingeniería de software, la inteligencia artificial y el Red Teaming:

- Ingenieros de Ciberseguridad y Red Teamers que desean integrar IA en sus flujos de trabajo sin exponer la telemetría de sus objetivos.
- Arquitectos de IA y Desarrolladores Backend interesados en la orquestación multi-agente (LangGraph, LangChain) y el paradigma del Model Context Protocol (MCP).
- DevSecOps y Administradores de Sistemas que requieren automatizar la auditoría de contenedores Docker y la remediación activa con Principio de Menor Privilegio (PoLP).
- CISOs y Líderes Técnicos que buscan adoptar modelos de lenguaje locales (como Gemma 4) garantizando el cumplimiento de normativas de privacidad absoluta (Zero-Telemetry).

Se asume un conocimiento intermedio de Python, Docker, redes y conceptos básicos de modelos de lenguaje.

Resumen

Vertex MCP Bible documenta la arquitectura y despliegue del Vertex Intelligence Core (VIC), un ecosistema de ciberseguridad autónoma que opera en dos planos complementarios. El primero, blindado y air-gapped, orquesta cinco microservicios en contenedores aislados mediante LangGraph para auditar infraestructuras Docker en Windows/WSL2 a través de puentes Socat y defenderse frente al OWASP LLM Top 10. El segundo —el plano soberano— saca el Core del contenedor para empuñar un arsenal dinámico de servidores MCP locales: un registro de servidores al estilo Claude Desktop, un cliente stdio que respeta el handshake completo del protocolo, y un motor de misiones locales donde el modelo sintetiza informes sobre resultados reales sin caer en la trampa del agente autónomo. El libro recorre además la evolución hacia el Red Teaming autónomo, la soberanía de datos en reposo y la portabilidad táctica en dispositivos Edge (ARM), y no oculta las trincheras: desde el nombre de host fantasma que rompe la comunicación hasta la disciplina de cadena de suministro frente al typosquatting de dependencias.

Introducción

La ciberseguridad ofensiva moderna y la automatización mediante Inteligencia Artificial ya no pueden depender de nubes centralizadas ni de arquitecturas monolíticas expuestas. Cada vez que un analista envía datos de un escaneo de red a una API comercial en la nube, está generando telemetría que puede ser interceptada, almacenada o perfilada. La soberanía operativa está en riesgo.

El Model Context Protocol (MCP) redefine la forma en que los agentes de IA interactúan con herramientas del sistema. Sin embargo, para que esta interacción sea segura, debe operar bajo un estricto esquema air-gapped. En esta introducción, establecemos las bases de por qué la arquitectura del Vertex Intelligence Core (VIC) abandona la dependencia de la nube. Exploraremos cómo la ejecución local de modelos neuronales, estrictamente enlazados mediante grafos de estado, garantiza cero fugas de datos corporativas.

A lo largo de este libro, descubriremos que la potencia de un sistema de ciberseguridad autónomo no radica meramente en la capacidad individual de sus herramientas de escaneo o remediación, sino en cómo se sincronizan sus agentes para generar una respuesta táctica, determinista y soberana. Bienvenido al futuro de la inteligencia artificial defensiva y ofensiva. Bienvenido a VIC.

Capítulo 1: El Despliegue del Ecosistema MCP y el Paradigma Air-Gapped

La ciberseguridad ofensiva moderna y la automatización mediante Inteligencia Artificial ya no pueden depender de nubes centralizadas ni de arquitecturas monolíticas expuestas. El Model Context Protocol (MCP) redefine la forma en que los agentes de IA interactúan con herramientas del sistema, bases de datos y motores de red bajo un estricto esquema air-gapped que garantiza cero fugas de telemetría corporativa.

1.1 La Filosofía de los 5 Nodos en Contenedores Aislados

Para construir un núcleo de inteligencia autónomo, modular y blindado, VIC (Vertex Intelligence Core) se descompone en una flota de microservicios Docker comunicados mediante una red privada interna (`mcp_secure_net`):

- **mcp-api-gateway (Puerto 8000):** Enrutamiento perimetral y control de tráfico. Tecnología subyacente: Traefik v3.0.
- **mcp-recon (Puerto 8001):** Escaneo de red y análisis de superficie de ataque. Tecnología: Python / Nmap.
- **mcp-vector (Puerto 8002):** Memoria persistente a largo plazo y RAG local. Tecnología: ChromaDB / Docker Volumes.
- **mcp-orchestrator (Puerto 8003):** Orquestación multi-agente y razonamiento. Tecnología: LangGraph / LangChain / Gemma 4.
- **mcp-infra (Puerto 8004):** Auditoría de seguridad de contenedores y host. Tecnología: Docker SDK / Psutil.

- **mcp-harden (Puerto 8005):** Remediación y autodefensa activa (Generación de parches). Tecnología: FastAPI / Python.

1.2 Las Trincheras de Ingeniería: Superando el Entorno Windows / WSL2

El despliegue de VIC sobre Windows exigió resolver restricciones críticas de arquitectura impuestas por el sistema de virtualización de Docker Desktop:

- **Persistencia Vectorial Segura:** ChromaDB requiere control absoluto sobre los ficheros SQLite subyacentes. Se implementó un esquema de volúmenes nativos gestionados combinados con la inyección de la asignación de dueños (chown) en tiempo de build dentro del Dockerfile, evitando fallos por permisos de usuario.
- **El Bypass del Socket de Docker:** Para auditar contenedores sin correr microservicios bajo el usuario root, se implementó un Puente Socat (TCP Proxy) en el punto de entrada (entrypoint). Este túnel redirige `/var/run/docker.sock` a un puerto local (`127.0.0.1:2375`), permitiendo que el demonio consuma la API mediante el comando `runuser` bajo el perfil restringido `vertex_mcp`.

1.3 Orquestación Cognitiva con LangGraph

El ciclo de vida de una auditoría en VIC no es lineal; es un grafo de estados dirigido por LangGraph. El pipeline encadena la exploración de puertos (recon), la recuperación de ChromaDB (vector), la inspección de riesgos de Docker (infra) y la síntesis deductiva final ejecutada por Gemma 4 localmente a través de LM Studio.

1.4 Topología Zero-Trust en mcp_secure_net

Un error de diseño en arquitecturas de microservicios es asumir que la simple existencia de una red interna de Docker blindaba automáticamente la infraestructura. En Vertex Coders LLC no confiamos en la red por defecto; implementamos un modelo de Zero-Trust a nivel de socket.

Dentro de mcp_secure_net, si un atacante lograra comprometer el nodo mcp-recon, teóricamente podría intentar escanear o exfiltrar datos de la base de datos vectorial en mcp-vector (puerto 8002). Para neutralizar esto, VIC implementa reglas de firewall internas (iptables) dentro de cada contenedor mediante el script de arranque, limitando la comunicación exclusivamente a las rutas autorizadas por el grafo de LangGraph:

```
# Limpiamos reglas existentes
iptables -F
# Bloqueamos todo el tráfico entrante por defecto
iptables -P INPUT DROP
iptables -P FORWARD DROP

# Permitimos tráfico local (loopback) para que Uvicorn y
# ChromaDB interactúen
iptables -A INPUT -i lo -j ACCEPT

# Permitimos conexiones entrantes SOLAMENTE desde el nodo
# orquestador
iptables -A INPUT -p tcp -s 172.20.0.5 --dport 8002 -j ACCEPT

# Rechazamos todo lo demás en la subred
iptables -A INPUT -i eth0 -j DROP
```

Con este bloqueo estructural, el sistema garantiza que incluso en el escenario de un Remote Code Execution (RCE) en un nodo perimetral, el atacante queda ciego frente al resto del ecosistema.

Capítulo 2: El Cerebro Neuronal

— Orquestación Multi-Agente con LangGraph

La potencia de un sistema de ciberseguridad autónomo no radica meramente en la capacidad individual de sus herramientas, sino en cómo se sincronizan sus agentes. En el núcleo de VIC, la orquestación ya no se confía a scripts lineales frágiles, sino a una máquina de estados dirigida mediante LangGraph.

2.1 Arquitectura del Estado Compartido (AgentState)

Cada tarea de auditoría comienza instanciando un diccionario tipado (AgentState) que viaja de forma fluida a través de los microservicios. Este estado centralizado recopila dinámicamente los vectores de ataque en un flujo secuencial estricto:

- `task`: El objetivo u orden táctica dictada por el operador.
- `recon_data`: Resultados estructurados del escaneo de red de superficie obtenidos por el nodo `mcp-recon`.
- `vector_data`: Contexto histórico y directrices de la empresa recuperadas por similitud semántica desde ChromaDB (`mcp-vector`).
- `infra_data`: Inventario de contenedores y nivel de exposición frente al demonio Docker auditado por `mcp-infra`.
- `final_report`: El informe sintético final redactado con razonamiento deductivo.

2.2 El Enlace Air-Gapped con LM Studio y Gemma 4

Para cumplir con el estándar de seguridad corporativo en cuanto a la soberanía de los datos, el nodo orquestador evita cualquier dependencia de APIs comerciales en la nube. Utiliza langchain-openai apuntando a un puente local directo hacia LM Studio:

```
llm = ChatOpenAI(
    base_url="http://
        host.docker.internal:1234/v1",
    api_key="lm-studio-local",
    temperature=0.3,
    model="gemma"
)
```

Esta configuración aísla por completo la telemetría de red y las vulnerabilidades internas. El modelo procesa los datos crudos de Nmap y los riesgos de contenedores ejecutándose como root, traduciéndolos en un informe táctico estructurado en Markdown limpio, con jerarquía ejecutiva, hallazgos técnicos y recomendaciones de remediación inmediata.

2.3 Ingeniería de Prompts Táctica para Gemma 4

Para evitar alucinaciones en modelos locales como Gemma 4, se implementa un System Prompt blindado. La estrategia consiste en Few-Shot Prompting cargado desde memoria estática en el contenedor, forzando salidas en JSON estricto y prohibiendo que el modelo asuma intenciones no explícitas en el AgentState.

2.4 Grafo de Decisiones Condicionales

LangGraph no solo avanza secuencialmente; usa aristas condicionales. Si mcp-recon detecta un puerto 8080 con un servicio vulnerable, el grafo ramifica el estado. En lugar de pasar directamente a la síntesis, enruta el flujo hacia un nodo intermedio que pide más contexto histórico a mcp-vector o genera un payload de prueba antes de consolidar el final_report.

Capítulo 3: Infraestructura Ofensiva y Remediación Activa (mcp-infra y mcp-harden)

Un sistema de inteligencia autónomo no puede limitarse a observar las fallas de un ecosistema; debe poseer la capacidad de auditar el sustrato físico y aplicar correcciones quirúrgicas en tiempo real.

3.1 La Interrogación del Demonio: El Puente Socat y el Bypass en Windows

Auditar el motor de contenedores desde una perspectiva de privilegios mínimos exige interactuar directamente con el socket de control (`/var/run/docker.sock`). Sin embargo, en entornos de desarrollo basados en Windows y WSL2, Docker Desktop aplica restricciones estrictas que bloquean la modificación de permisos.

VIC implementa un Puente Socat (TCP Proxy) en el script de arranque:

1. **Capa de Arranque:** Un proceso maestro inicializa el contenedor con capacidades de red y monta el socket de Docker.
2. **Traducción de Red Local:** El comando `socat` crea un túnel seguro que mapea el socket protegido a un puerto TCP interno exclusivo (`127.0.0.1:2375`).
3. **Degradación de Privilegios:** Inmediatamente después, el proceso cede el control operativo al usuario restringido `vertex_mcp`, ejecutando el servidor Uvicorn sobre una conexión local aislada.

3.2 Auditoría de Riesgos y el Principio de Menor Privilegio (PoLP)

Una vez establecido el enlace, el nodo mcp-infra interroga dinámicamente el estado de la flota para identificar configuraciones críticas:

- Contenedores ejecutándose bajo el usuario root (identificados como "security_risk": true).
- Uso indebido de la bandera --privileged.
- Exposición de bases de datos críticas (Postgres, Redis) en redes perimetrales no segmentadas.

3.3 El Motor de Autodefensa: mcp-harden

Cuando VIC detecta una imagen mal configurada o un contenedor expuesto, entra en acción el nodo mcp-harden. Este microservicio genera un nuevo Dockerfile optimizado:

```
# --- VERTEX CODERS SECURE DOCKERFILE TEMPLATE ---
FROM postgres:16-alpine

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

RUN useradd -m -s /bin/bash vertex_secure_user
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN chown -R vertex_secure_user:vertex_secure_user /app

USER vertex_secure_user
```

3.4 Validación de Parches mediante Sandboxing Dinámico

Cuando mcp-harden genera un nuevo Dockerfile, no se aplica a ciegas. VIC lanza un contenedor efímero de pruebas usando el nuevo Dockerfile, ejecuta un script de validación (linting y health-check) y, solo si pasa la prueba, lo marca como "Ready for Production" en el reporte final para el operador.

Capítulo 4: Blindaje y Mitigación de Amenazas — OWASP LLM Top 10

La integración de LLMs con herramientas de ejecución del sistema operativo introduce vectores de ataque avanzados. VIC implementa directrices estrictas basadas en el marco OWASP LLM Top 10.

4.1 Inyección de Prompts y Manipulación de Herramientas

Un atacante puede intentar secuestrar el flujo de razonamiento del agente inyectando instrucciones maliciosas en los datos devueltos por el nodo de reconocimiento. VIC neutraliza este riesgo implementando una separación estricta de contextos:

- Los datos crudos se tratan estrictamente como cadenas de texto no ejecutables dentro de las variables de estado.
- El modelo Gemma 4 opera con una temperatura baja (0.3) y un system prompt endurecido que restringe alterar la misión original.

4.2 Autenticación Criptográfica entre Microservicios

La comunicación horizontal entre los cinco nodos exige un control perimetral mediante cabeceras de autorización obligatorias:

```
headers \= {  
  "Content-Type": "application/json",  
  "X-MCP-Access-Token":  
    "vertex-super-secret-key-2026"
```

```
}
```

La función centralizada `verify_mcp_token` actúa como un middleware de seguridad en cada microservicio FastAPI, rechazando cualquier intento de invocación carente del token.

4.3 Gestión de Salidas Inseguras y Principio de Menor Privilegio

El análisis de código estático y la generación de parches siguen el principio de defensa en profundidad. Ninguna salida generada por el agente se aplica en el sistema anfitrión sin la validación previa del operador humano (esquema *human-in-the-loop*), evitando RCE derivado de alucinaciones.

4.4 Defensa contra Envenenamiento de Datos en ChromaDB

Abordando el OWASP LLM04 (Data Poisoning), VIC implementa firmas hash en los vectores insertados en ChromaDB. Si un documento histórico de auditoría es modificado manualmente por un atacante con acceso físico, el hash no coincidirá y el nodo `mcp-vector` rechazará el contexto comprometido en tiempo de recuperación (RAG).

Capítulo 5: Automatización Operativa y Scripts de Despliegue

La gestión de un ecosistema de microservicios distribuido en contenedores Docker sobre un sistema operativo anfitrión Windows exige una capa de automatización robusta.

5.1 El Patrón de Orquestación por Scripts de Control

VIC utiliza scripts de Python puro (como `build_infra.py`, `fix_socket_socat.py`) para realizar operaciones de forma programática:

- Generación Dinámica de Dockerfiles.
- Modificación Quirúrgica de Configuraciones YAML sin corromper la sintaxis.
- Secuencias de Reconstrucción Atómica (`docker-compose build` y `up -d --force-recreate`).

5.2 Estandarización del Código y Limpieza de Perfiles

Cada script de automatización de VIC opera bajo estrictas directrices de calidad:

- Codificación UTF-8 Forzada (`encoding="utf-8"`, `newline="\n"`).
- Manejo Explícito de Excepciones con respuestas JSON-RPC 2.0.
- Principio de Menor Privilegio en Tiempo de Ejecución.

5.3 El Motor de Logging Forense Air-Gapped

Para registrar la actividad de los agentes sin enviar telemetría a la nube, un script de Python captura los flujos de AgentState y los consolida en un fichero SQLite local estructurado, ideal para análisis forense posterior en caso de un incidente de seguridad.

Capítulo 6: El Futuro de VIC — Hacia la Autonomía Operativa y OracleAI

El desarrollo y consolidación de VIC marca un antes y un después en la forma en que Vertex Coders LLC concibe la automatización de la ciberseguridad. La soberanía tecnológica y la seguridad air-gapped son totalmente viables.

6.1 La Evolución hacia OracleAI

El siguiente peldaño es la convergencia de VIC con OracleAI. Mientras VIC actúa como el núcleo táctico, OracleAI absorberá estas capacidades para escalar hacia la predicción de cadenas de suministro, la toma de decisiones logísticas complejas y el análisis financiero automatizado.

6.2 Próximos Pasos en la Arquitectura de Agentes

- Orquestación Multi-Nodo Distribuida en clústeres Docker Swarm o Kubernetes aislados.
- Memoria Episódica Avanzada para perfiles de riesgo adaptativos.
- Agentes de Autonomía Corrección (Self-Healing) con pipelines de integración continua.

6.3 Interfaz de Migración hacia OracleAI

Se diseñará un contrato de API (JSON-RPC adaptado) que permitirá a VIC enviar métricas de ciberseguridad a la base de datos de OracleAI sin romper el paradigma air-gapped, implementando un canal de un solo sentido (*data diode* lógico) en la red.

Capítulo 7: Red Teaming

Autónomo — Del Reconocimiento a la Explotación

Un auditor pasivo solo raspa la superficie; un motor de inteligencia ofensiva debe ser capaz de validar las vulnerabilidades en tiempo real.

7.1 Generación de Payloads Asistida por IA

Cuando mcp-recon detecta un servicio web, el AgentState enruta los datos crudos hacia Gemma 4. El LLM evalúa cabeceras y versiones, y genera un script de explotación (ej. curl para probar SSRF o Path Traversal) bajo consentimiento explícito.

```
def generate_payload_node(state: AgentState) -> AgentState:
    recon_data := state["recon_data"]
    prompt := f"""
    Eres un analista de Red Team. Analiza
    estos datos de Nmap: {recon_data}
    Si detectas un servidor web, genera
    un comando 'curl' para probar
    una vulnerabilidad de Path Traversal.
    Responde SOLO con el comando bash.
    """
    payload :=
        llm.invoke(prompt).content.strip()
    state["generated_payload"] := payload
    return state
```

7.2 Simulación de Movimiento Lateral (Mapa de Rutas)

Si mcp-infra detecta un contenedor expuesto, Gemma 4 traza un "mapa de rutas de pivotaje". Analiza variables de entorno inyectadas y dependencias de red, construyendo un árbol de amenazas teórico para que el operador comprenda el impacto de un posible RCE.

Capítulo 8: Zero-Telemetry y Soberanía de Datos — El Archivo del Saber

El paradigma Air-Gapped no se limita a aislar la red externa; debe asegurar que el dato permanezca blindado en reposo y en memoria.

8.1 La Criptografía en el Vector Local

La base de datos ChromaDB se encuentra cifrada en reposo. Utilizando volúmenes encriptados nativos de Docker (o LUKS en el host anfitrión), se garantiza que si la máquina anfitriona Windows es robada o decomisada, el histórico de auditorías y vectores de clientes corporativos no sea legible sin las claves de descifrado operadas por el usuario `vertex_mcp`.

8.2 Depuración de Memoria RAM del LLM

Los modelos locales tienden a retener en caché datos sensibles. VIC incorpora técnicas de limpieza de memoria entre sesiones de LangGraph, forzando el reinicio del contexto del modelo y purgando el histórico de conversaciones inmediatamente después de consolidar el `final_report`, mitigando el riesgo de volcado de memoria RAM (RAM Scraping) por parte de malware en el host.

Capítulo 9: VIC en el Edge — Red Team Físico y Despliegues Portátiles

La verdadera versatilidad de un núcleo blindado es su capacidad de operar fuera del laboratorio, directamente en el campo.

9.1 Migración a Arquitecturas ARM

Los 5 microservicios Docker han sido adaptados para correr en dispositivos Edge de bajo consumo (Raspberry Pi 4/5 o NVIDIA Jetson Nano). Esto permite a un operador de Vertex Coders llevarse el Vertex Intelligence Core en el bolsillo, enchufarlo a la red del cliente mediante un cable de red o WiFi aislado, y ejecutar auditorías autónomas sin depender de servidores on-premise del cliente.

9.2 Bypass de Controles de Red Físicos

Cuando el cliente impone controles como 802.1X (NAC) o MAC Filtering, VIC implementa scripts de Python para el spoofing de direcciones MAC de los contenedores, suplantando dispositivos legítimos (como impresoras o cámaras IP) para burlar defensas perimetrales locales de forma automática en la fase de reconocimiento inicial.

Capítulo 10: El Backend Soberano

— Cuando el Core Sale del Contenedor

Durante los primeros nueve capítulos, el Vertex Intelligence Core vivió enteramente dentro de Docker: seis nodos blindados, una red interna `mcp_secure_net` y un orquestador que jamás tocaba el sistema anfitrión. Ese aislamiento era una virtud. Pero la evolución del Core hacia un operador de herramientas dinámicas nos obligó a romper una frontera que creíamos sagrada: la del contenedor.

Este capítulo documenta esa ruptura. No fue una decisión estética, sino una imposición de la física del sistema operativo.

10.1 El Muro de Cristal: Windows contra el Contenedor Linux

La nueva generación de servidores MCP de Vertex no vive en la red interna de Docker. Vive en el host, y se declara con rutas físicas del anfitrión Windows:

```
{
  "vertex-cyber-mcp": {
    "command":
      "G:\\Astra\\vertex-cyber-mcp\\.venv\\Scripts\\python.exe",
    "args": ["-m", "vertex_cyber_mcp.server"]
  },
  "tello": {
    "command": "H:\\mcp-drone\\.venv\\Scripts\\python.exe",
    "args": ["H:\\mcp-drone\\tello_server.py"]
  }
}
```

Aquí está el muro de cristal: el orquestador de VIC corre dentro de un contenedor **Linux**. Un contenedor Linux no tiene una unidad G:

ni H:, y no puede ejecutar un binario .exe de Windows. Por más que el grafo reciba la orden de invocar vertex-cyber-mcp, es ciego y manco frente a este arsenal. El demonio de Docker vive en su propia realidad, y las rutas del operador humano no existen en ella.

La conclusión arquitectónica es ineludible: **para empuñar herramientas que viven en el host, el proceso que las invoca debe correr en el host.** No hay puente de red que resuelva una diferencia de sistema operativo a nivel de sistema de ficheros y de formato ejecutable.

10.2 La Migración: del Contenedor Efímero al Proceso Soberano

La solución fue extraer el `oracle_backend` de su contenedor y ejecutarlo de forma nativa sobre Windows con Uvicorn. En el `docker-compose.oracle.yml`, este servicio publicaba su API en el puerto 8010:

```
oracle-core:
  build:
    context: ../oracle_backend
  container_name: oracle-core-api
  ports:
    - "8010:8010"
```

Al sacarlo del contenedor surgió el primer roce de trinchera: el contenedor `oracle-core-api` seguía vivo, acaparando el puerto 8010. En Windows, Uvicorn con `SO_REUSEADDR` puede reportar que "levantó" en un puerto ya ocupado por Docker sin fallar de inmediato, dejando dos procesos peleándose las conexiones de forma impredecible. La lección: antes de levantar el backend soberano, se baja el contenedor, o se le asigna un puerto limpio (8015). El backend soberano corre sobre el intérprete Python nativo del anfitrión, con visión completa de G: y H: y capacidad de lanzar los .exe del arsenal.

10.3 La Trinchera del Nombre Fantasma: host.docker.internal contra localhost

El backend soberano seguía necesitando hablar con el grafo dockerizado (el orquestador en el puerto 8003). Cuando el backend vivía dentro de Docker, lo alcanzaba mediante un nombre especial:

```
vic_orchestrator_url = "http://host.docker.internal:8003/api/v1/mcp/orchestrate/execute"
```

Al lanzar la primera misión desde el backend ya migrado, el sistema devolvió un error contundente:

```
[!] Fallo en la misión: Error de comunicación con VIC: [Errno 11004] getaddrinfo failed
```

getaddrinfo failed es un fallo de resolución de nombre, y el culpable es host.docker.internal. Ese nombre es un fantasma: **solo existe dentro de un contenedor Docker**, donde Docker Desktop lo inyecta apuntando al anfitrión. Desde un proceso nativo de Windows, ese nombre no resuelve a nada. Y no hace falta: desde el propio host, el orquestador ya está publicado en localhost:8003 gracias al mapeo de puertos del contenedor.

La corrección, blindada para servir en ambos mundos mediante una variable de entorno con valor por defecto seguro:

```
# host.docker.internal solo resuelve DENTRO de un contenedor.  
# Con el backend  
# soberano en el host, el orquestador se alcanza por  
# localhost.  
orchestrator_host = os.getenv("VIC_ORCHESTRATOR_HOST",  
                              "localhost")  
vic_orchestrator_url = f"http://{orchestrator_host}:8003/api/v1/mcp/orchestrate/execute"
```

Un detalle mínimo — el nombre de un host — separaba una misión exitosa de un fallo total. Así son las trincheras de la

ingeniería distribuida: el error no está en la lógica, sino en el mapa de la red que cambió bajo nuestros pies.

10.4 El Core como Despachador Soberano (BFF)

Con la migración, el `oracle_backend` dejó de ser un simple proxy del grafo y asumió un rol superior: el de **despachador soberano** (un patrón Backend-for-Frontend con inteligencia de enrutamiento). Ante cada orden, el Core decide el plano de ejecución:

- **Misión de auditoría de infraestructura** → la reenvía al grafo dockerizado (8003), que opera en su burbuja air-gapped sobre `mcp_secure_net`.
- **Invocación de una herramienta local** → la ejecuta él mismo, en el host, empuñando los servidores stdio del arsenal del operador.

VIC pasó así de tener un único plano a tener dos, complementarios. El plano dockerizado conserva su blindaje para el reconocimiento de red y la auditoría de contenedores. El plano soberano aporta la capacidad de orquestar cualquier herramienta que el operador registre en su máquina. El contenedor dejó de ser una prisión y se convirtió en uno de dos brazos de un mismo cerebro.

Lección de trinchera: el aislamiento por contenedor es un escudo, no un dogma. Cuando la misión exige tocar el suelo del sistema anfitrión, el ingeniero soberano sabe cuándo salir de la burbuja — y cómo hacerlo sin perder el blindaje del resto de la flota.

Capítulo 11: El Registro MCP Dinámico — Servidores stdio al Estilo Claude Desktop

Un backend soberano capaz de ejecutar herramientas del host no sirve de nada si no sabe qué herramientas existen. VIC necesitaba un registro: un catálogo vivo de servidores MCP que el operador pudiera declarar, activar y sincronizar sin recompilar nada. La respuesta fue adoptar un estándar que la industria ya había consolidado.

11.1 El Formato Universal: mcpServers.json

El ecosistema MCP converge en un formato de declaración de servidores que usan tanto Claude Desktop como LM Studio: un objeto `mcpServers` donde cada entrada define un `command`, sus `args` y, opcionalmente, sus variables de entorno (`env`):

```
{
  "mcpServers": {
    "mi-calculadora": {
      "command": "node",
      "args": ["H:/mcp-calculator/server.js"]
    },
    "vertex-cyber-mcp": {
      "command": "...python.exe",
      "args": ["-m", "vertex_cyber_mcp.server"],
      "env": {
        "VCMCP_ALLOWED_ROOTS": "...",
        "VCMCP_ENABLE_SEMGREP": "true"
      }
    }
  }
}
```

Adoptar este formato no fue un capricho de compatibilidad: significa que cualquier servidor MCP que el operador ya use en Claude Desktop o LM Studio funciona en VIC sin modificación. El registro es portable por diseño.

11.2 El Gestor MCP: Registro Visual y Sincronización

Sobre ese formato, VIC construyó el **Gestor MCP**, una vista del frontend Angular con dos paneles: un editor del `mcpServers.json` y una lista de servidores detectados, cada uno con un interruptor de activación. Un botón —"Sincronizar con Core"— envía al backend únicamente los servidores encendidos.

El contrato es deliberadamente simple. El frontend filtra los activos y envía `{ mcpServers: {...} }`; el backend los extrae y los guarda:

```
def sync_servers(self, config_payload: dict) -> int:
    self.active_servers = config_payload.get("mcpServers", {})
    # ... persistencia en disco ...
    return len(self.active_servers)
```

11.3 La Trinchera del Sync Fantasma

Durante la integración descubrimos que el botón "Sincronizar con Core" mentía. El componente registraba la intención en consola y mostraba un mensaje de éxito... pero la llamada HTTP real estaba comentada como un `// TODO`. El botón solo ejecutaba un `alert()`. Los servidores jamás salían del navegador, y del lado del backend `get_active_servers()` devolvía siempre un diccionario vacío.

Es una trampa clásica: una interfaz que reporta éxito sobre una operación que nunca ocurrió. La corrección exigió inyectar el servicio HTTP en el componente y activar el envío real. La lección: un "éxito" en la UI no prueba nada; solo el dato viajando de extremo a extremo lo hace.

11.4 Persistencia: Sobrevivir al Reinicio

La segunda trinchera fue más silenciosa. Los servidores sincronizados vivían solo en la memoria del proceso backend. Cada reinicio —y en desarrollo hay muchos— vaciaba el registro, obligando a re-sincronizar a mano. El propio motor de síntesis, tras un reinicio, ejecutaba misiones contra un catálogo vacío y reportaba "servidor no activo" sobre herramientas que el operador creía cargadas.

El registro ya escribía un fichero `mcpServers_active.json` en cada sincronización, pero nunca lo leía al arrancar. El cierre del círculo fue trivial y definitivo: cargar ese fichero en la construcción del servicio.

```
def _load_from_disk(self) -> None:
    try:
        if os.path.exists(CONFIG_PATH):
            with open(CONFIG_PATH, "r", encoding="utf-8") as f:
                data = json.load(f)
                if isinstance(data, dict):
                    self.active_servers = data
    except Exception:
        self.active_servers = {}
```

Lección de trinchera: el estado que solo vive en RAM es estado que se pierde. Un registro operativo debe sobrevivir al reinicio, o dejará de ser una fuente de verdad para convertirse en una fuente de sorpresas.

Capítulo 12: El Cliente MCP Real

— El Handshake que lo Cambió Todo

Tener el registro de servidores era solo la mitad. La otra mitad —la que de verdad costó sangre— era hablarles bien. Un servidor MCP no es un script al que se le tira una línea y responde; es un participante de un protocolo con su propio ritual de apertura. Ignorar ese ritual fue el error que nos tuvo trancados.

12.1 El Executor que Hablaba "Medio MCP"

La primera versión del executor de VIC hacía lo siguiente: lanzaba el proceso, le escribía un único mensaje `tools/list` por la entrada estándar, y usaba `communicate()` para cerrar el canal y esperar a que el proceso muriera.

El problema es que un servidor MCP conforme a la especificación no funciona así, por dos razones:

1. **Exige un handshake previo.** Antes de aceptar cualquier `tools/list` o `tools/call`, el servidor espera una secuencia de apertura. Pedirle herramientas en frío se ignora o se rechaza.
2. **Es un proceso persistente.** No termina tras un mensaje; se queda vivo escuchando. Un `communicate()` que espera su muerte se queda esperando hasta el `timeout`.

El síntoma era un cuelgue de quince segundos y una respuesta vacía. El executor hablaba "medio MCP": la sintaxis del mensaje era correcta, pero el protocolo estaba incompleto.

12.2 El Handshake del Protocolo stdio

El diálogo MCP real sobre stdio sigue una coreografía estricta sobre una conexión que permanece abierta:

```
initialize          → (el servidor responde sus
    capacidades)
notifications/initialized → (el cliente confirma)
tools/list | tools/call → (recién ahora, operaciones)
```

Reconstruir este baile a mano es posible, pero frágil: el framing de los mensajes, el manejo de la conexión persistente y la concurrencia son fáciles de romper.

12.3 La Trinchera del Typosquat: Disciplina de Supply-Chain

La decisión correcta fue apoyarse en el SDK oficial de MCP en vez de reimplementar el protocolo. Pero al ir a instalarlo, un simple `pip install --dry-run mcp` encendió una alarma. La resolución de dependencias arrastraba paquetes con nombres inquietantes:

```
Collecting httpx2>=2.5.0
Collecting httpcore2==2.13.0
```

`httpx2` y `httpcore2` no son las librerías HTTP conocidas —esas se llaman `httpx` y `httpcore`, sin el "2" al final. Añadir un dígito a un nombre popular es un patrón clásico de *typosquatting*, la técnica con la que se cuela código malicioso en cadenas de dependencias.

No confirmamos que fueran maliciosas. Pero tampoco pudimos verificarlas como legítimas, y en un proyecto de ciberseguridad esa incertidumbre basta: ante la duda, la dependencia no entra. La verificación reveló que esos paquetes aparecían solo en la línea 2.x del SDK; la línea 1.x estable usaba `httpx` y `httpcore` normales, ya presentes en el entorno. La solución fue fijar la versión:

```
mcp>=1.28,<2
```

Con ese anclaje desaparecieron los paquetes sospechosos y quedó la línea estable, la misma API que usan Claude Desktop y LM Studio por dentro.

La ironía es la mejor lección del capítulo: estábamos haciendo a mano, sobre nuestra propia máquina, exactamente el chequeo de cadena de suministro que VIC está diseñado para automatizar. La doctrina de la herramienta se aplicó a la construcción de la herramienta.

12.4 El SDK Oficial y la Herencia del Entorno

Con el SDK anclado, el ejecutor se convirtió en un cliente MCP real. El handshake y el ciclo de vida del proceso quedaron en manos de la librería:

```
async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()
        result = await session.call_tool(tool_name, arguments=
            arguments)
```

Quedaba un último detalle de trinchera, propio de Windows: los servidores stdio necesitan el entorno del sistema (el PATH, entre otras variables) para que sus binarios arranquen. Pasarles solo las variables personalizadas los dejaba sin contexto. La solución fue heredar el entorno completo del anfitrión y superponer las variables específicas de cada servidor.

La prueba de fuego fue deliberadamente humilde: invocar la herramienta factorial de la calculadora con el argumento 5. La respuesta —120— no tenía nada de espectacular, salvo que probaba que el handshake completo había ocurrido: proceso lanzado, protocolo negociado, herramienta invocada, resultado devuelto. El cliente por fin hablaba MCP entero.

Lección de trinchera: un protocolo se respeta completo o no se respeta. Hablar "medio protocolo" produce fallos silenciosos que parecen bugs de red pero son bugs de contrato. Y cuando una dependencia huele mal, la disciplina de no instalarla vale más que la prisa de compilar.

Capítulo 13: Misiones Locales — Herramientas y Síntesis sin Agente

Con un cliente MCP funcional, VIC podía por fin ejecutar el arsenal del operador. La pregunta ya no era técnica sino de diseño: ¿quién decide qué herramientas usar y cómo se convierten sus resultados en inteligencia? La respuesta definió el carácter del sistema.

13.1 El Rechazo del Agente Autónomo

La tentación evidente era construir un agente: dejar que el modelo decidiera qué herramienta llamar, en un bucle de razonamiento. Se descartó, y por razones concretas, no dogmáticas.

VIC nació precisamente para independizarse de las limitaciones de ejecutar herramientas dentro de LM Studio, donde las peticiones largas revientan por timeout. Un agente reintroducía ese mismo problema por otra puerta: sobre una GPU modesta, cada vuelta de razonamiento del modelo local toma minutos, y un bucle de agente encadena varias. Peor aún, un modelo de tamaño medio puede elegir la herramienta equivocada o pasarle argumentos torcidos. Para una herramienta de seguridad, eso es lo inaceptable: lentitud e imprevisibilidad. El determinismo vale más que la magia.

13.2 El Pipeline Determinístico: Ejecutar, luego Sintetizar

El diseño elegido separa tajantemente la ejecución del razonamiento. El backend ejecuta las herramientas que el operador seleccionó, recoge sus resultados, y hace **una sola** llamada al modelo para redactar el informe. El modelo no ejecuta nada; solo analiza lo que ya está masticado.

```
async def run_local_mission(task_id, body):
    results = []
    for inv in body.get("invocations", []):
        res = await MCPProcessExecutor.call_tool(
            inv["server_name"], active[inv["server_name"]],
            inv["tool_name"], inv.get("arguments", {})
        )
        results.append(res)
    report = await synthesize_report(body.get("task", ""),
                                     results)
    # ... guardar en TASKS_DB ...
```

La síntesis reutiliza la voz de VIC y llama a Gemma por el endpoint local, con una consigna explícita: basarse únicamente en los resultados entregados, sin inventar datos ausentes.

13.3 Grounding: Gemma Analiza, No Inventa

La validación fue reveladora. Se ejecutaron tres herramientas estadísticas sobre una serie de tiempos de respuesta con un valor claramente anómalo, y se le pidió al modelo detectar la anomalía. El informe usó los números exactos que las herramientas devolvieron —media 78.4, desviación 130.81— y señaló el valor 340 como anómalo, cuantificándolo como 4.33 veces la media. Nada inventado; todo derivado de datos reales.

Aún más revelador fue el caso de fallo. En una corrida donde los servidores no estaban sincronizados, las herramientas devolvieron errores. En lugar de inventar latencias para cumplir con la

consigna, el modelo reportó honestamente la ausencia de datos y correlacionó los fallos como un problema de disponibilidad. Ese comportamiento —negarse a alucinar cuando no hay datos— es exactamente lo que se le exige a una herramienta de seguridad, donde un reporte confiado sobre datos fantasma es peor que ningún reporte.

13.4 Asincronía y Sondeo

La síntesis local es lenta por naturaleza: el modelo tarda minutos. Bloquear la petición HTTP durante ese tiempo la condenaría al timeout. Por eso la misión local adopta el mismo patrón asíncrono que el grafo dockerizado: el backend encola la tarea, devuelve un identificador al instante, y el frontend sondea el estado (QUEUED → RUNNING → COMPLETED) hasta recibir el informe. La infraestructura de tareas y sondeo se reutilizó sin cambios: dos planos de ejecución, un solo mecanismo de seguimiento.

Lección de trinchera: la autonomía no es un fin en sí mismo. Un pipeline determinístico donde el humano elige las herramientas y el modelo solo razona sobre resultados reales es más rápido, más predecible y más honesto que un agente que decide solo. En seguridad, saber exactamente qué se ejecutó vale más que la elegancia de que la máquina lo adivine.

Capítulo 14: Epílogo — El Manifiesto de Vertex Coders

Hemos llegado al final de esta edición de la Vertex MCP Bible, pero en las trincheras de la ingeniería de élite, un sistema vivo nunca se detiene. Lo que comenzó como una idea para estructurar agentes autónomos locales se ha convertido en un ecosistema robusto, blindado y soberano.

14.1 Las Lecciones de Trinchera

- **La persistencia no se regala:** La gestión de bases de datos vectoriales exige disciplina estricta de permisos y asignación de propietarios antes de degradar usuarios.
- **El sistema se respeta:** Romper la barrera del socket de Docker en Windows mediante el puente Socat demostró que con ingenio arquitectónico es posible mantener la postura de menor privilegio.
- **La IA debe ser soberana:** Un modelo local, orquestado a través de LangGraph, puede auditar redes y contenedores sin enviar un solo byte a la nube.

14.2 El Legado para el Futuro

Este libro y el código que lo sustenta quedan como el estándar operativo oficial para Vertex Coders LLC. Cada script en Python, cada microservicio en FastAPI, cada regla de hardening y cada nodo de nuestro grafo representa horas de combate técnico resueltas con elegancia y precisión.

La soberanía de la inteligencia artificial y la automatización de la ciberseguridad ofensiva pertenecen a quienes se atreven a construir sus propias herramientas desde cero.

El núcleo está activo. La flota está blindada. Continuamos operando.

Fin de la Vertex MCP Bible.

Publicado por Denis Sanchez Leyva — CEO de Vertex Coders LLC (Miami, 2026).

Referencias y Marco de Trabajo

El desarrollo y despliegue del Vertex Intelligence Core se sustenta en los siguientes estándares, tecnologías y marcos de referencia técnicos:

- OWASP LLM Top 10: Marco de referencia para la mitigación de vulnerabilidades en Aplicaciones de Modelos de Lenguaje Grande (Inyección de Prompts, Envenenamiento de Datos, etc.).
- LangGraph & LangChain: Framework de orquestación para la construcción de máquinas de estados y grafos de decisiones en flujos multi-agente.
- Docker SDK & Traefik v3.0: Tecnologías subyacentes para la contenerización, segmentación de red y enrutamiento perimetral (Zero-Trust).
- ChromaDB: Base de datos vectorial de código abierto para la recuperación aumentada (RAG) local.
- LM Studio & Gemma 4: Entorno de inferencia local y modelo de lenguaje de código abierto utilizado para el razonamiento deductivo sin conexión externa.
- Principio de Menor Privilegio (PoLP) y Defense in Depth: Doctrinas de ciberseguridad aplicadas a la degradación de permisos (runuser) y la validación de salidas (human-in-the-loop).

Sobre el Autor

Denis Sanchez Leyva es el CEO y fundador de Vertex Coders LLC, con sede en Miami, Florida. Es un arquitecto de software e investigador de ciberseguridad especializado en la convergencia entre la Inteligencia Artificial autónoma, la infraestructura de contenedores blindados y el Red Teaming táctico.

Con una filosofía centrada en la soberanía de los datos y la ingeniería de trinchera, Denis lidera el desarrollo del Vertex Intelligence Core (VIC) y la plataforma central OracleAI. Su trabajo se enfoca en demostrar que la automatización de la ciberseguridad y la privacidad de los datos no son conceptos mutuamente excluyentes, y que la verdadera potencia de la IA radica en su capacidad para operar de manera aislada, determinista y soberana.